CYBERSECURITY TRACK • PYTHON MODULE

Assignment 5

Multithreaded Port Scanner (Python)

Done by: Abdulhadi Al-Ahdal

1. Project Overview

The objective of this assignment is to build an efficient network port scanner using Python to identify active services on a target IP address. By leveraging the standard `socket` module along with multithreading via `concurrent.futures`, the tool rapidly inspects a range of network ports to determine their status accurately without unnecessary execution delays.

2. Development Steps

1 **Socket Function Creation**

Define a dedicated scanner worker function that accepts a target IP and port number, creating an IPv4 TCP socket instance per check.

2 **Timeout Configuration**

Configure an explicit socket connection timeout to prevent thread blocking when inspecting closed or filtered ports.

3

4

Parallel Execution

Utilize thread pool management from `concurrent.futures` to spawn concurrent threads for accelerated scanning.

Port Mapping & Reporting

Iterate across target ports (e.g., 1–1024), reporting open ports instantly alongside total benchmarked execution time.

3. Technical Features & Utilities

ThreadPoolExecutor Management: Configured with `max_workers = 50` to handle concurrent network requests smoothly.

Non-blocking Probe Method: Utilizes `connect_ex()`, which returns `0` upon success rather than throwing runtime exceptions.

4. Implementation Source Code

`port_scanner.py`

Python 3.x

```
import socket
import time
from concurrent.futures import ThreadPoolExecutor

def scan_port(target_ip, port):
    try:
        s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
        s.settimeout(1.0)
```

```
            result = s.connect_ex((target_ip, port))
            if result == 0:
                print(f"[+] Port {port:5d} is OPEN")
            s.close()
        except Exception:
            pass

    def main():
        target = input("Enter target IP or hostname: ").strip()
        if not target:
            target = "127.0.0.1"

        try:
            target_ip = socket.gethostbyname(target)
        except socket.gaierror:
            print("[-] Invalid hostname or IP address.")
            return

        print("-" * 50)
        print(f"Scanning target: {target_ip}")
        print("Port range     : 1 - 1024")
        print(f"Scan started at: {time.strftime('%Y-%m-%d %H:%M:%S')}")
        print("-" * 50)

        start_time = time.time()

        with ThreadPoolExecutor(max_workers=50) as executor:
            for port in range(1, 1025):
                executor.submit(scan_port, target_ip, port)

        end_time = time.time()
        elapsed = end_time - start_time
```

```
    print("-" * 50)
    print(f"Scan completed in {elapsed:.2f} seconds.")

if __name__ == "__main__":
    main()
```

5. Acceptance Criteria

- ✓ **High Precision & Speed**
  Successfully discovers open and closed ports with minimal scan duration across the entire 1-1024 range.

- ✓ **Fault Tolerance & Stability**
  Ensures zero script crashes during unresponsive target checks while handling socket cleanup properly.